

RAPPORT DE TP SYSTEME EMBARQUE

Préparé par :
Mohamed Aderak

09 Mai 2025

**Centre d'excellence IT
Pr. MENSOU Sara**

Table des Matières :

1.Introduction

2.Exercice 1 : Gestion des Feux Tricolores

- **2.1 Montage sous Proteus**
- **2.2 Programme Arduino**
- **2.3 Résultats et Analyse**

3.Exercice 2 : Communication Série SPI

- **3.1 Identification du Protocole**
- **3.2 Montage sous Proteus**
- **3.3 Code Arduino (Maître et Esclaves)**
- **3.4 Analyse des Signaux avec l'Oscilloscope**
- **3.5 Extension avec Potentiomètre et LEDs**

4.Conclusion

5.Annexes

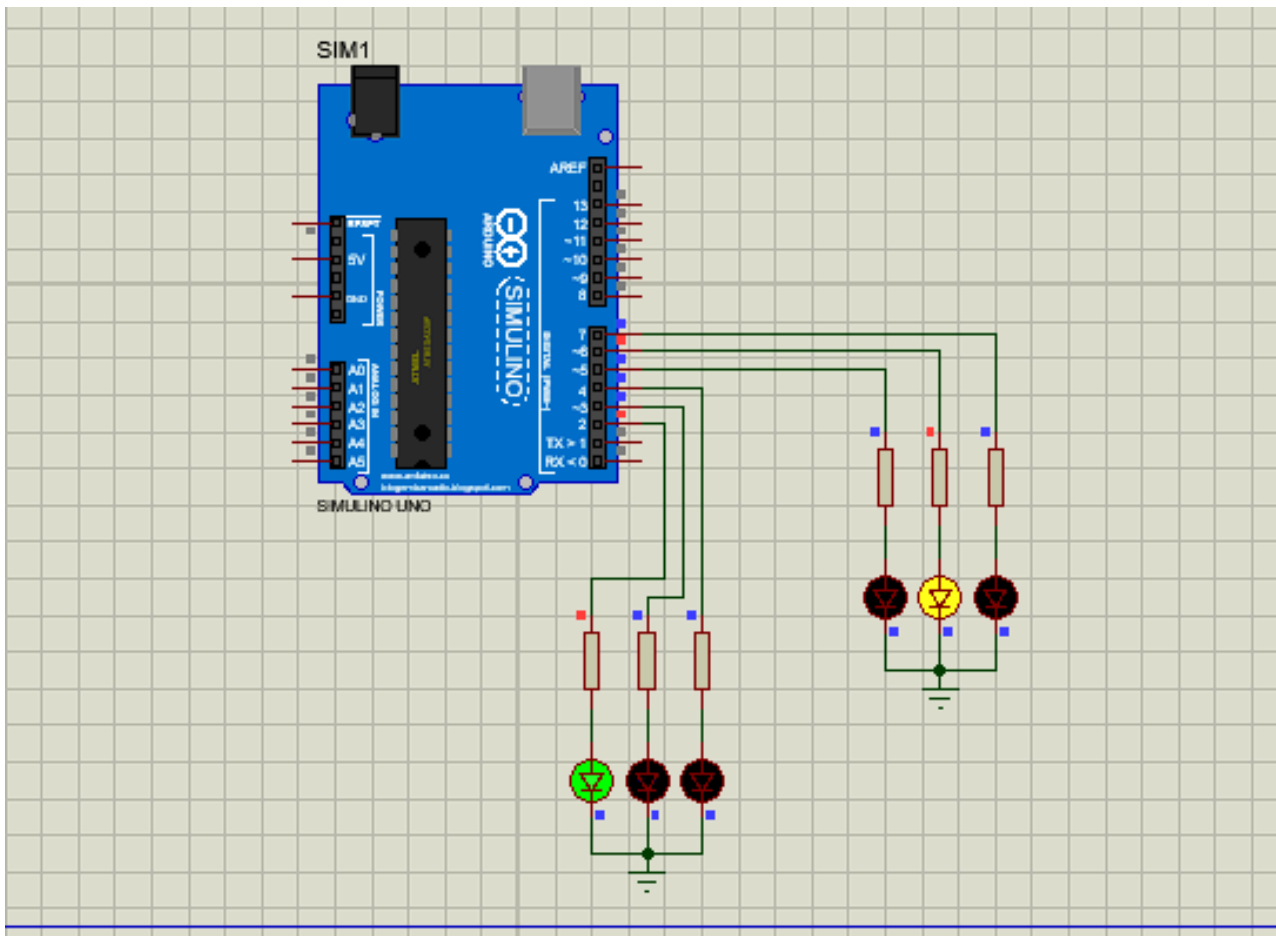
- **Codes Sources Complets**
- **Captures d'Écran**

Introduction

Ce rapport présente la réalisation de deux exercices en systèmes embarqués utilisant la carte Arduino Uno et le logiciel Proteus. Le premier exercice simule un système de feux tricolores, tandis que le second explore la communication SPI entre un maître et des esclaves.

Exercice 1 : Gestion des Feux Tricolores

2.1 Montage sous Proteus



- Deux groupes de LEDs (rouge, jaune, vert) connectés aux broches 2-7 de l'Arduino.
- Résistances de 220 Ω en série avec chaque LED.

2.2 Programme Arduino

```
1  const int R1 = 2, J1 = 3, V1 = 4;
2  const int R2 = 5, J2 = 6, V2 = 7;
3
4  int tempsVert = 5000;
5  int tempsJaune = 2000;
6  int tempsRouge = 5000;
7
8  void setup() {
9      pinMode(R1, OUTPUT);
10     pinMode(J1, OUTPUT);
11     pinMode(V1, OUTPUT);
12     pinMode(R2, OUTPUT);
13     pinMode(J2, OUTPUT);
14     pinMode(V2, OUTPUT);
15
16     digitalWrite(R1, LOW);
17     digitalWrite(J1, LOW);
18     digitalWrite(V1, LOW);
19     digitalWrite(R2, LOW);
20     digitalWrite(J2, LOW);
21     digitalWrite(V2, LOW);
22 }
```

```
void loop() {
    //----- Phase 1 : Groupe 1 Vert | G
    digitalWrite(V1, HIGH);
    digitalWrite(R2, HIGH);
    delay(tempsVert);

    // Transition Groupe 1 : Vert → Jaune
    digitalWrite(V1, LOW);
    digitalWrite(J1, HIGH);
    delay(tempsJaune);

    //----- Phase 2 : Groupe 1 Rouge | G
    digitalWrite(J1, LOW);
    digitalWrite(R1, HIGH);
    digitalWrite(R2, LOW);
    digitalWrite(V2, HIGH);
    delay(tempsVert);

    // Transition Groupe 2 : Vert → Jaune
    digitalWrite(V2, LOW);
    digitalWrite(J2, HIGH);
    delay(tempsJaune);

    // Réinitialiser pour le cycle suivant
    digitalWrite(J2, LOW);
    digitalWrite(R1, LOW);
}
```

2.3 Résultats et Analyse

- **Chronogramme** : Respect du cycle vert → jaune → rouge (Figure 2).
- **Modularité** : Les variables tempsVert, tempsJaune, et tempsRouge peuvent être ajustées pour adapter le système au trafic.

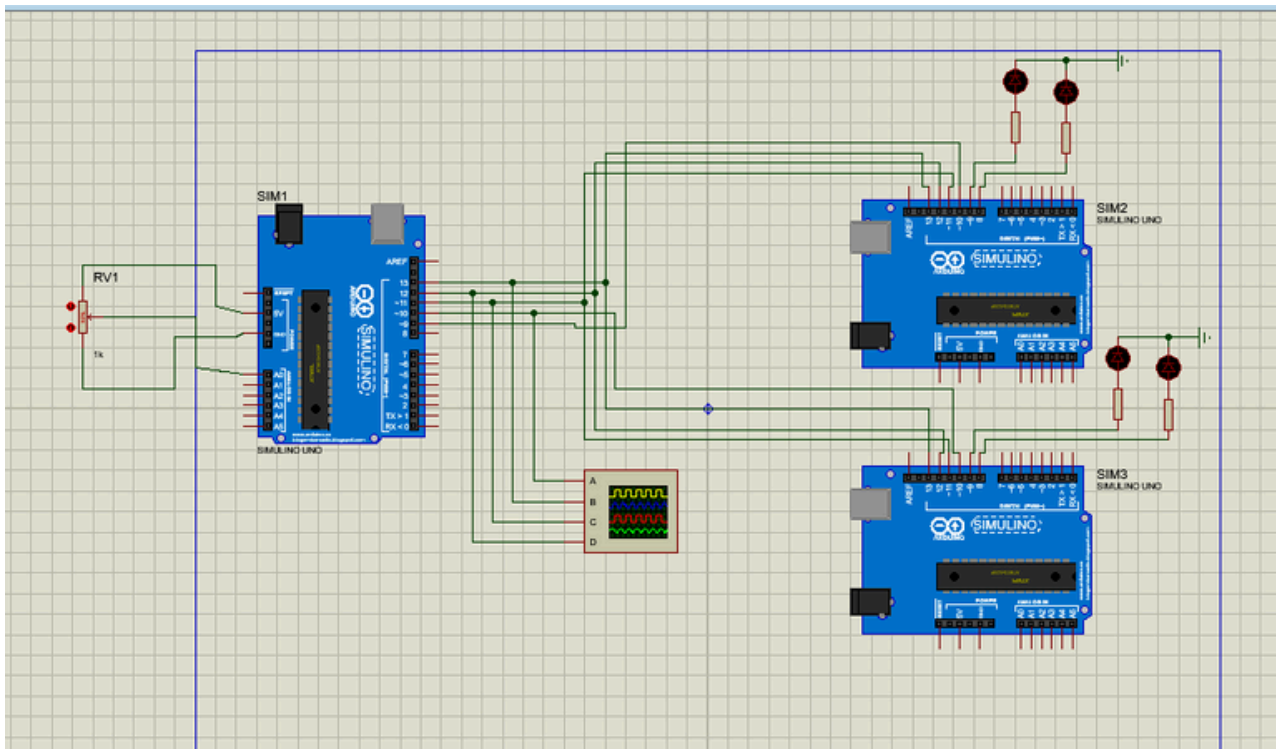
Exercice 2 : Communication Série SPI

3.1 Identification du Protocole

Le protocole utilisé est SPI (Serial Peripheral Interface), identifié par les signaux SCK, MOSI, MISO, et SS.

3.2 Montage sous Proteus

- **Maître (Carte 1) :** Connecté aux esclaves via les broches SPI (SCK:13, MOSI:11, MISO:12, SS:10 et 9).
- **Esclaves (Cartes 2 et 3) :** LEDs sur les broches 8 et 9.



3.3 Code Arduino

Code_Maitre.ino

```
1  #include <SPI.h>
2
3  void setup() {
4      SPI.begin();
5      pinMode(10, OUTPUT);
6      pinMode(9, OUTPUT);
7  }
8
9  void loop() {
10     digitalWrite(10, LOW);
11     SPI.transfer('A');
12     digitalWrite(10, HIGH);
13     delay(500);
14
15     digitalWrite(9, LOW);
16     SPI.transfer('B');
17     digitalWrite(9, HIGH);
18     delay(500);
19 }
```

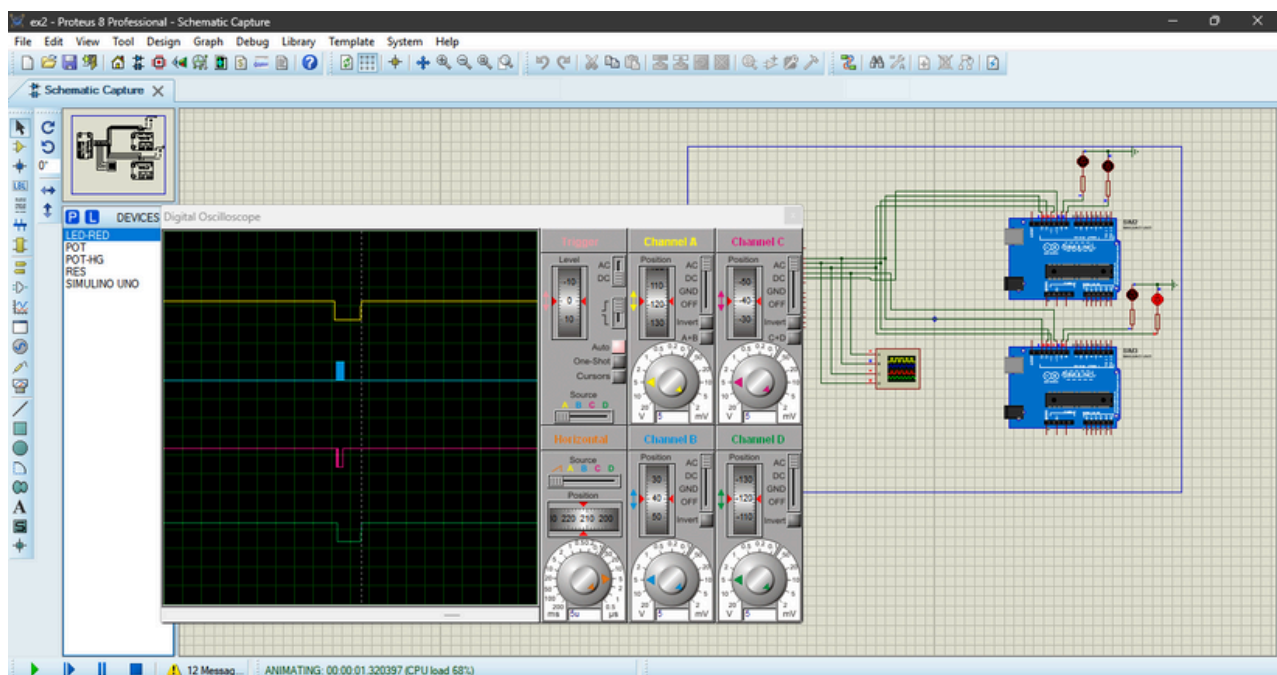
Code_Esclave.ino

```
1  #include <SPI.h>
2  volatile char receivedData;
3
4  void setup() {
5      pinMode(MISO, OUTPUT);
6      SPCR |= _BV(SPE);
7      SPI.attachInterrupt();
8      pinMode(8, OUTPUT);
9      pinMode(9, OUTPUT);
10 }
11
12 ISR(SPI_STC_vect) {
13     receivedData = SPDR;
14
15     if(receivedData == 'A') {
16         digitalWrite(8, !digitalRead(8));
17     }
18     else if(receivedData == 'B') {
19         digitalWrite(9, !digitalRead(9));
20     }
21 }
22
23 void loop() {}
```

Maître (Carte 2):

Esclave (Carte 2&3):

3.4 Analyse des Signaux avec l'Oscilloscope



- **SS** : Passe à LOW pour sélectionner un esclave.
- **SCK** : Horloge à 1 MHz.
- **MOSI** : Données binaires (ex: 0b01000001 pour 'A').

Durée d'un bit :

$$1 / 1 \text{ MHz} = 1\mu\text{s}$$

3.5 Extension avec Potentiomètre et LEDs

- **Code Esclave :**

Code_Esclave_V2.ino

```

1  #include <SPI.h>
2  volatile char receivedData;
3
4  void setup() {
5      pinMode(10, INPUT);
6      SPCR |= _BV(SPE);
7      SPI.attachInterrupt();
8      pinMode(8, OUTPUT);
9      pinMode(9, OUTPUT);
10 }
11
12 void loop() {
13
14 }
15
16 ISR(SPI_STC_vect) {
17     receivedData = SPDR;
18     if (receivedData == 'C') {
19         digitalWrite(8, !digitalRead(8));
20         digitalWrite(9, !digitalRead(9));
21     } else if (receivedData == 'D') {
22         digitalWrite(8, LOW);
23         digitalWrite(9, LOW);
24     }
25 }
```

- **Code Maître :**

Code_Maitre_V2.ino

```

1  #include <SPI.h>
2
3  void setup() {
4      SPI.begin();
5      pinMode(10, OUTPUT);
6      pinMode(9, OUTPUT);
7      digitalWrite(10, HIGH);
8      digitalWrite(9, HIGH);
9  }
10
11 void loop() {
12     int potValue = analogRead(A0);
13     float voltage = potValue * (5.0 / 1023);
14
15     if (voltage > 3.0) {
16         digitalWrite(10, LOW);
17         SPI.transfer('C');
18         digitalWrite(10, HIGH);
19     } else {
20         digitalWrite(10, LOW);
21         SPI.transfer('D');
22         digitalWrite(10, HIGH);
23     }
24     delay(200);
25 }
```

Conclusion

Les deux exercices ont permis de maîtriser la simulation de systèmes embarqués avec Arduino et Proteus. La communication SPI a été validée par l'oscilloscope, et les feux tricolores fonctionnent selon le cahier des charges.

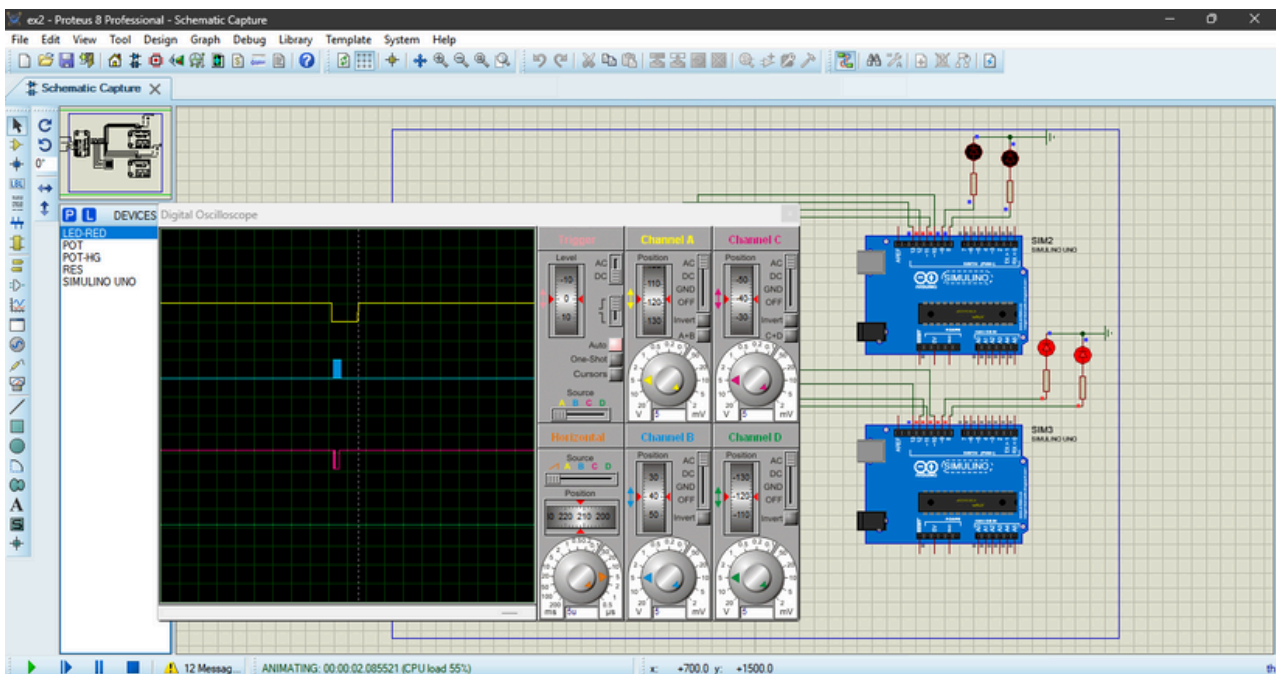
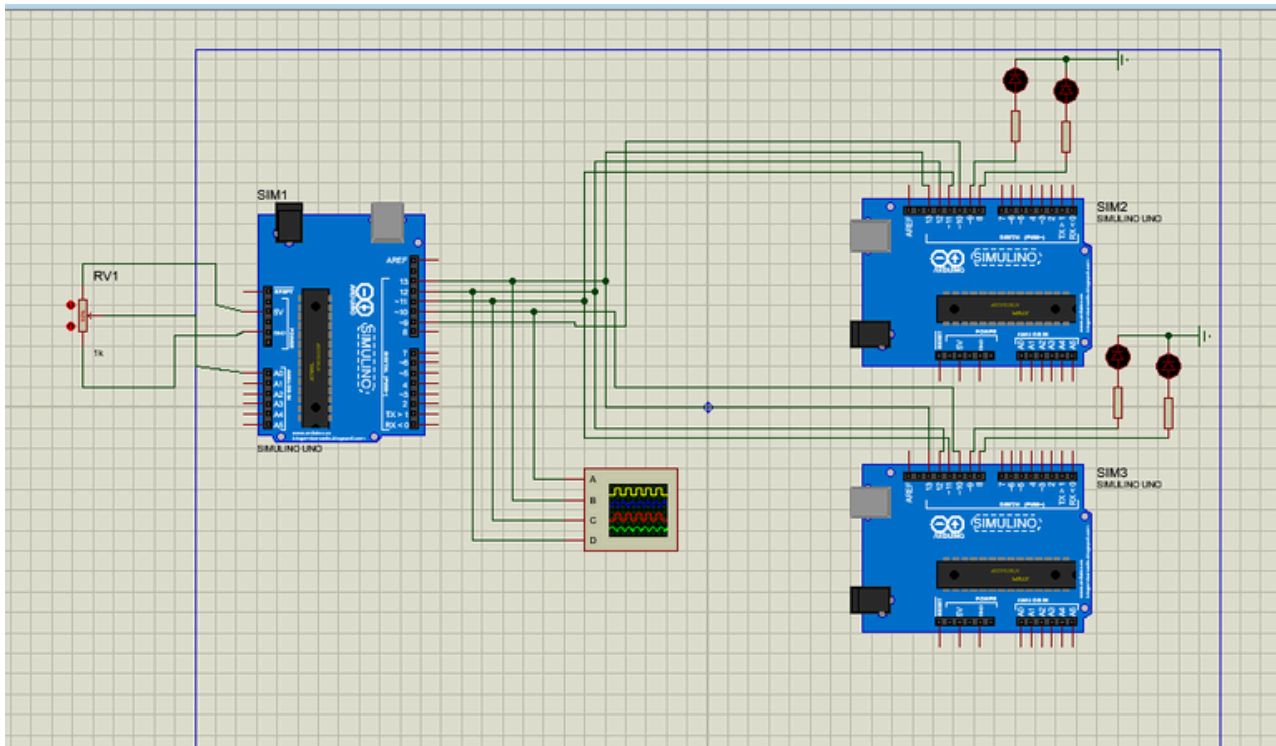
Annexes

- **Code Complet : Voir Github repo.**
- **Captures d'Écran :**
 - **Montage des feux tricolores.**
 - **Signaux SPI sur l'oscilloscope.**
 - **Montage avec potentiomètre et LEDs.**

Voir sur Github

`github*com/MohamedAderak/Embedded-Systems-Lab-IISE-S6`

remplacer (*) avec une point (.)



CENTER
D'EXCELLENCE IT

Préparé par :
**Mohamed
Aderak**

09-05-2025

Université Ibn Zohr

